

yolo5_ws 上手指南

yolo5_ws 上手指南

修订记录

第一章 工程结构

1.1 model

1.1.1 asamu_degrees90.wts

1.1.2 best_appasamu.wts

1.2 notROS-tools

1.2.1 分类种类确定

1.2.2编译转换工具

1.2.3 TensorRT 模型文件的更新

1.2.4 修改识别程序

1.3 src

第二章 使用

与基于行为树的抓取demo配合使用

第三章 其他

输入

修订记录

版本号	修订日期	修订人员	修订内容

第一章 工程结构

- ├── model #模型文件
- ├── notROS-tools #模型转换工具
- └── src #核心功能包

1.1 model

Important

出厂内置模型为best_appasamu.wts

1.1.1 asamu_degrees90.wts

本模型用于识别：农夫山泉、橙汁、阿萨姆奶茶

标签分别为:农夫山泉:0,橙汁:1,阿萨姆奶茶:2

```
nongfu: 0
orangeate: 1
asamu: 2
```

1.1.2 best_appasamu.wts

本模型用于识别：农夫山泉、橙汁、阿萨姆奶茶、橘子、苹果

标签分别为:农夫山泉:0,橙汁:1,阿萨姆奶茶:2, 橘子:3,苹果:4

```
nongfu: 0
orangeate: 1
asamu: 2
orange: 3
apple: 4
```

1.2 notROS-tools

本工具用于转换模型，从而实现用GPU完成推理的过程

1.2.1 分类种类确定

根据预使用的模型的标签数量，修改CLASS_NUM

```
~/yolo5_ws$ gedit ./notROS-tools/tensorrtx-yolov5-v6.0/yolov5/yololayer.h
static constexpr int CLASS_NUM = 3;#修改为识别物体种类的数量
#eg
## 使用 asamu_degrees90.wts则CLASS_NUM = 3
## 使用 best_appasamu.wts则CLASS_NUM = 5
```

1.2.2编译转换工具

```
~/yolo5_ws$ cd ./notROS-tools/tensorrtx-yolov5-v6.0/yolov5/ && mkdir build && cd build
cmake ..
make -j$((nproc - 1))
```

以上指令生成了一个 yolov5 的可执行程序以及一个 libmyplugins.so 的库文件。至此转换工具已就绪,开始进行模型转换操作。将model文件中需要使用的模型拷贝到yolov5/build 目录执行下面命令:

```
sudo ./yolov5 -s name.wts name_s.engine s # 根据实际情况替换name
# eg
## sudo ./yolov5 -s best_appasamu.wts best_appasamu_s.engine s
```

Important

必须使用 sudo 提权!

等待一段时间即可获取 TensorRT 模型文件 model_s.engines,此为最终模型文件。

1.2.3 TensorRT 模型文件的更新

将 **model_s.engines** 和 **libmyplugins.so** 两个文件放入到工控机端的视觉识别模块中的 tensorRT 目录即可完成替换,路径参考如下:

```
/yolo5_ws/src/yolo5_detector/scripts/tensorRT
```

1.2.4 修改识别程序

以模型asamu_degrees90_s.engines为例:

```
~/yolo5_ws$ gedit src/yolo5_detector/scripts/yolo5rt_detector.py
```

更具实际情况修改以下参数

```
pluginlibname = "libmyplugins.so"
wfname="best_appasamu_s.engine" # tensorRT识别模型文件
categories = ["nongfu","orangeate","asamu","orange","apple"] #识别的类别，和网络同步
```

1.3 src

编译功能包

Catkin make 时 注意-DPYTHON_LIBRARY 的位置,使用 whereis libpython3.6m.so.1.0 查看真实目录。

```
catkin_make -DPYTHON_EXECUTABLE=/usr/bin/python3 -
DPYTHON_INCLUDE_DIR=/usr/include/python3.6 -DPYTHON_LIBRARY=/usr/lib/x86_64-linux-
gnu/libpython3.6m.so.1.0
#em 与empty 同名需要卸载em 而且empty==3.3.4
```

第二章 使用

与基于行为树的抓取demo配合使用

基于行为树的抓取demo具体使用方式请参考其readme

在每次进入检测叶节点前设置ros参数服务器中参数/pick_target_type为1.1小节中对应的标签即可如

```
rosparam set /pick_target_type 4
```

当然也可以在对应的启动文件中写入参数已指定默认抓取目标如

```
<param name="/pick_target_type" type="int" value="4"/>
```

第三章 其他

输入

默认输入话题应为640*480 大小的图像，如需其他大小则应该

1. 根据实际情况修改

```
#1.处理色彩和深度数据
depth_image = np.frombuffer(depthImage.data, dtype=np.uint16).reshape(480, 640)
color_image = np.frombuffer(colorImage.data, dtype=np.uint8).reshape(480, 640, 3)
```

以正确的将其转换为open所接受的格式

2. 更改预测时当前图像大小，为实际大小

```
#预测一帧
results, use_time = yolov5_wrapper.infer(cimg)
#def infer(self, raw_image, img_originHW=(480,640)):
```